



Level 1 – Foundation

Duração estimada: 14 semanas · 5-10h/semana

Pré-requisito: Já conhece Node/TS, Git e lógica de programação

Foco: HTTP/HTTPS, API Design, SQL, SOLID, testes, Docker, autenticação, CI/CD

Projeto principal: Task Manager API



Skills ao final do nível

- [] **HTTP/HTTPS:** ciclo requisição-resposta, métodos, status codes, headers, TLS handshake, HTTP/2 vs HTTP/1.1
 - [] **API Design:** RESTful naming, versionamento, paginação (cursor vs offset), HATEOAS, formatação de erros (RFC 7807)
 - [] **SQL:** modelagem relacional, JOINS, índices, transações ACID, migrations, window functions
 - [] **Big O:** análise de complexidade – $O(1)$, $O(n)$, $O(n^2)$, $O(\log n)$, $O(n \log n)$
 - [] **Clean Code:** nomes, funções pequenas, tratamento de erros consistente, DRY, KISS
 - [] **SOLID:** SRP, OCP, LSP, ISP, DIP na prática
 - [] **Design Patterns:** Repository, Factory, Adapter
 - [] **Testes unitários:** Jest, mocks, stubs, spies, TDD, cobertura
 - [] **Autenticação:** JWT, bcrypt, refresh tokens, middleware de auth, OWASP básico
 - [] **Docker:** multi-stage, docker-compose, volumes, networks, healthcheck
 - [] **CI/CD:** GitHub Actions – pipeline de lint → test → build → push
 - [] **Algoritmos:** busca binária, recursão, sorting (quick, merge), two pointers
 - [] **Logging:** logs estruturados (pino/winston), níveis de log
 - [] **Graceful Shutdown:** SIGTERM, draining connections, cleanup
-



Semana a Semana

SEMANA 1 – HTTP/HTTPS: Como a Web Funciona

Teoria (5h)

HTTP: - Ciclo: cliente abre conexão TCP → envia request → servidor processa → response → fecha - Métodos: `GET` (seguro, idempotente), `POST` (não-idempotente), `PUT` (idempotente), `PATCH`, `DELETE`, `OPTIONS`, `HEAD` - Idempotência: `GET`, `PUT`, `DELETE` são idempotentes (mesmo efeito se chamado N vezes); `POST` e `PATCH` não são - Status codes por categoria: - **1xx** – informacional (101 = switching protocols – WebSocket) - **2xx** – sucesso (200 OK, 201 Created, 204 No Content) - **3xx** – redirecionamento (301 moved permanently, 302 found, 304 not modified) - **4xx** – erro do cliente (400 bad request, 401 unauthorized, 403 forbidden, 404 not found, 409 conflict, 422 unprocessable, 429 too many requests) - **5xx** – erro do servidor (500 internal, 502 bad gateway, 503 service unavailable, 504 gateway timeout) - Headers importantes: - **Request:** `Authorization`, `Content-Type`, `Accept`, `User-Agent`, `Cache-Control`, `If-Modified-Since`, `If-None-Match` - **Response:** `Content-Type`, `Content-Length`, `Cache-Control`, `ETag`, `Location`, `Set-Cookie`, `WWW-Authenticate` - **CORS:** `Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`, `Access-Control-Allow-Headers` - **Segurança:** `Strict-Transport-Security`, `X-Content-Type-Options`, `X-Frame-Options`, `Content-Security-Policy`

HTTPS/TLS: - Handshake TLS 1.3: ClientHello → ServerHello + Certificate → Key Exchange → Secure Connection - Certificados: CA, chain of trust, self-signed vs CA-signed - Por que HTTPS é obrigatório: confidencialidade, integridade, autenticação

HTTP/2 vs HTTP/1.1: - Multiplexing (várias requests na mesma conexão) - Server push, header compression (HPACK) - Binary framing layer

Prática (4h)

```
# Veja o handshake HTTP
curl -v https://api.github.com

# Veja os headers de resposta
curl -I https://api.github.com

# Teste métodos HTTP
curl -X POST https://jsonplaceholder.typicode.com/posts \
  -H "Content-Type: application/json" \
  -d '{"title":"foo","body":"bar","userId":1}'

# Veja o TLS handshake
openssl s_client -connect api.github.com:443 -tls1_3

# Teste cache com ETag
curl -I -H "If-None-Match: \"...\"" https://api.github.com
```

 **Exercício:** Crie `http-fundamentos.md` com: - Tabela de métodos (idempotente? seguro? com/sem body?) - Tabela de status codes (categoria

+ exemplos) - Diagrama textual do TLS handshake - Diferenças HTTP/1.1 vs HTTP/2 vs HTTP/3

SEMANA 2 – API Design Patterns

 **Teoria (4h)** - Leia: [Microsoft REST API Guidelines](#) - Leia: [Google API Design Guide](#) - Assista: [Hussein Nasser – API Design](#)

Tópicos: - **Naming:** usar substantivos (`/users`, `/products/:id`), NUNCA verbos (`/getUsers`, `/createProduct`) - **Plurais:** `/users`, `/products/:id/reviews` - **Versionamento:** URL (`/v1/users`), header (`Accept: application/vnd.api+json;version=1`), query param - **Paginação:** - **Offset-based:** `?page=2&limit=20` – simples, mas problemático com dados em tempo real - **Cursor-based:** `?cursor=eyJpZCI6MTB9&limit=20` – mais robusto, performance constante - **Filtros:** `?status=pending&priority=high` - **Sorting:** `?sort=created_at:desc,name:asc` - **Campos parciais:** `?fields=id,name,email` (sparse fieldsets) - **Error handling (RFC 7807):** `json { "type": "https://api.exemplo.com/errors/validation", "title": "Erro de validação", "status": 422, "detail": "0 campo email é obrigatório", "errors": [ { "field": "email", "message": "Email inválido" } ] }`

 **Prática (4h)** - Projete a API da Task Manager seguindo RESTful best practices - Escreva o contrato OpenAPI/Swagger antes de implementar

SEMANA 3 – SQL: Modelagem e Consultas

 **Teoria (3h)** - Assista: [Fábio Akita – Modelagem de Dados](#) - Estude: normalização (1FN, 2FN, 3FN), chaves primárias/estrangeiras, índices, `EXPLAIN ANALYZE`

 **Prática (5h)**

```
-- Modele: users, tasks, categories
-- Com INSERT, SELECT, JOIN, GROUP BY, subqueries
-- Crie índices e analise com EXPLAIN ANALYZE
-- Use transações (BEGIN/COMMIT/ROLLBACK)
```

 **Exercício:** Modele o banco da Task Manager (users + tasks) – migrations SQL, queries de CRUD, índices apropriados.

SEMANA 4 – SQL Avançado: Janelas, CTEs, Performance

 **Teoria (3h)** - Window Functions: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD`, `NTILE` - CTEs (WITH): recursivas e não-recursivas - `EXPLAIN ANALYZE`
na prática: sequential scan vs index scan vs bitmap scan

Prática (4h)

```
-- Ranking de usuários por tasks concluídas
SELECT
  u.name,
  COUNT(t.id) as tasks_done,
  ROW_NUMBER() OVER (ORDER BY COUNT(t.id) DESC) as rank,
  LAG(COUNT(t.id)) OVER (ORDER BY COUNT(t.id) DESC) as prev_count
FROM users u
JOIN tasks t ON t.user_id = u.id AND t.status = 'done'
GROUP BY u.id;
```

SEMANA 5 – Big O + Algoritmos Essenciais

 **Teoria (4h)** - Leia: **Entendendo Algoritmos** – Cap. 1 a 4 e 6 a 7 -
Estude: Big O para cada estrutura (array, linked list, hash map, tree) -
Técnicas: two pointers, sliding window, recursão, memoização

Prática (4h)

```
// Implemente e analise complexidade:
binarySearch(arr, target)      // O(log n)
quickSort(arr)                 // O(n log n)
isPalindrome(str)              // O(n)
fibonacci(n)                   // O(2^n) vs O(n) com memo
twoSum(nums, target)           // O(n) com hash map
```

 **Exercício:** Resolva 5 fáceis no LeetCode. Para cada: anote a complexidade e **explique por quê**.

SEMANA 6 – Clean Code + SOLID + Error Handling

 **Teoria (4h)** - Leia: **Código Limpo** – Caps 1 a 6 (nomes, funções, comentários, formatação) - Leia: [SOLID com exemplos \(Refactoring Guru\)](#) -
Estude: **Error handling patterns** em APIs

Prática (5h)

```
// Error handling padronizado (RFC 7807)
class AppError extends Error {
  constructor(
    public statusCode: number,
    public type: string,
    public detail: string,
    public errors?: Array<{ field: string; message: string }>
  ) {
    super(detail);
  }
}

// Separação por camadas (SRP)
// Controller → Service → Repository

// Dependency Injection (DIP)
class TaskService {
  constructor(
    private repo: ITaskRepository,
    private logger: ILogger,
    private notify: INotificationService
  ) {}
}
```

 **Exercício:** Refatore um código RUIM (com tudo misturado) aplicando SRP + DIP. Documente antes/depois.

SEMANA 7 – Testes Unitários + TDD

 **Teoria (3h)** - Leia: [Martin Fowler – Mocks Aren't Stubs](#) - Estude: pirâmide de testes (unitário > integração > e2e), o que testar (regras de negócio, não frameworks)

Prática (5h)

```
// TDD: implementa validação de task
// 1. Escreve teste (RED)
// 2. Implementa (GREEN)
// 3. Refatora (REFACTOR)

// Mocks semânticos (não acoplar implementação)
const mockRepo = { save: jest.fn().mockResolvedValue(task) };
const service = new TaskService(mockRepo);

// Spies para verificar chamadas
expect(mockRepo.save).toHaveBeenCalledWith(
  expect.objectContaining({ title: 'Estudar' })
);
```

 **Exercício:** Testes para service de autenticação (register, login, refresh, token expirado, senha errada). Coverage ≥80%.

SEMANA 8 – Autenticação JWT + Refresh Token

📖 **Teoria (3h)** - Leia: [JWT.io](https://jwt.io) - Estude: access vs refresh token, bcrypt salt rounds, OWASP (armazenamento seguro de tokens, XSS, CSRF)

🔧 Prática (5h)

```
// Hash de senha (bcrypt: salt rounds = 10-12)
// JWT: payload mínimo (userId, role), expiresIn curto (15min)
// Refresh token: longa duração (7d), armazenado no banco, revogável
// Middleware de auth: verifica token, decodifica, injeta req.user

POST /auth/register  → { name, email, password }
POST /auth/login     → { email, password } → { accessToken, refreshToken }
POST /auth/refresh   → { refreshToken } → { accessToken, refreshToken }
POST /auth/logout    → revoga refresh token
```

SEMANA 9 – Docker + Docker Compose

📖 **Teoria (3h)** - Assista: [TechWorld with Nana – Docker](#) - Leia: [Docker Best Practices](#) - Estude: layers, multi-stage, `.dockerignore`, healthcheck, restart policies, networks

🔧 Prática (5h)

```
# Dockerfile multi-stage com usuário não-root
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS production
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/package*.json ./
RUN npm ci --production && npm cache clean --force
USER node
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=5s CMD wget -qO- http://localhost:3000/health
CMD ["node", "dist/index.js"]
```

```
# docker-compose.yml
services:
  app:
    build: .
    ports: [ "3000:3000" ]
    depends_on:
      db: { condition: service_healthy }
    env_file: .env
    networks: [ backend ]

  db:
    image: postgres:16-alpine
    volumes: [ pgdata:/var/lib/postgresql/data ]
    healthcheck:
      test: [ "CMD-SHELL", "pg_isready -U postgres" ]
      networks: [ backend ]

volumes: { pgdata: }
networks: { backend: }
```

SEMANA 10 – CI/CD + Logging + Graceful Shutdown

Teoria (2h)

CI/CD: - GitHub Actions: eventos (push, PR, schedule), jobs, steps, matrix, services, artifacts, secrets

Logging Estruturado: - Logs em JSON (não texto puro) – parseáveis por máquina - Níveis: `debug`, `info`, `warn`, `error`, `fatal` - Informações mínimas: timestamp, level, message, requestId, userId, serviceName - `pino` (mais rápido) vs `winston`

Graceful Shutdown:

```
process.on('SIGTERM', async () => {
  logger.info('Sinal SIGTERM recebido, desligando...');
  server.close(() => logger.info('Servidor HTTP fechado'));
  await db.close();
  await redis.disconnect();
  process.exit(0);
});
```

Prática (5h)

```

name: CI
on: [push, pull_request]
jobs:
  lint: ...
  test: ... (com services: postgres)
  build: ...
  docker-push:
    needs: [lint, test, build]
    runs-on: ubuntu-latest
    steps:
      - uses: docker/login-action@v3
      - uses: docker/build-push-action@v5

```

```

// Logging estruturado (pino)
const logger = pino({
  level: process.env.LOG_LEVEL || 'info',
  transport: { target: 'pino-pretty' }, // dev only
});
logger.info({ userId: 1, action: 'login' }, 'Usuário logou');

```

SEMANAS 11-12 – 🚀 Projeto: Task Manager API

Stack: Node/TS + PostgreSQL + Docker + Jest + Swagger

Arquitetura: Clean Architecture (domain → application → infrastructure)

```

src/
├── domain/      → entities + use-cases (PURPOS, sem imports externos)
├── application/ → interfaces (repositories, services)
├── infrastructure/ → implementações (Postgres, JWT, Express, Pino)
└── main/       → DI setup, server, graceful shutdown

```

Funcionalidades: - CRUD de tarefas com paginação cursor-based + filtros - Autenticação JWT + refresh token - Validação Zod em todos inputs - Error handling padronizado (RFC 7807) - Logs estruturados (pino) - Graceful shutdown - Docker multi-stage + compose - CI/CD (lint → test → build → push) - Swagger em `/api-docs`

SEMANA 13 – Revisão + Refatoração Guiada

- Pegue o código e aplique: Repository Pattern, error handling consistente, logs
 - Code review em si mesmo: funções >20 linhas? nomes ruins? SRP violado?
 - Otimize SQL queries com `EXPLAIN ANALYZE`
-

SEMANA 14 – Prova de Nível

1. **HTTP:** Qual a diferença entre 401 e 403? Quando usar 201 vs 200?
2. **HTTPS:** Explique o TLS handshake em 3 passos
3. **API Design:** Vantagens de cursor-based pagination sobre offset-based?
4. **SQL:** Escreva uma query com window function + CTE
5. **Clean Code:** Dê 3 violações de SOLID e como corrigir
6. **Docker:** Explique multi-stage build (por que usar? economiza o quê?)
7. **Big O:** Complexidade de busca em hash table vs lista ligada?
8. **JWT:** Como funciona refresh token? Proteção contra XSS?

 **Passa se:** Responde $\geq 6/8$ + projeto Task Manager completo no GitHub.

Temas Além do Código

Tema	Por que é importante	Onde usar
HTTP/HTTPS	Base de toda comunicação web	Todo projeto
TLS/SSL	Segurança, certificados, mão na roda com Nginx	Setup de servidores
API Design	APIs bem projetadas são mais fáceis de manter e consumir	Todo backend
Pagination	Offset vs cursor – impacto em performance	Listas grandes
Error Handling (RFC 7807)	Padrão de indústria para erros	APIs REST
Graceful Shutdown	Evita perda de dados, conexões pendentes	Produção
Logging Estruturado	Debugging, monitoramento, correlação	Produção
Idempotência	Segurança em retentativas (pagamentos, etc.)	APIs críticas

Próximo nível: `level-02-intermediate.md`